Simulación N-Body 2D con PCAM y OpenMP

Integrantes: David Domínguez (23712), Jonathan Diaz (23873) y Luis Padilla (23663)
Repositorio: https://github.com/Jonialen/labPCAM

Entorno: WSL2 Ubuntu 24.04; AMD Ryzen 5 4600H with Radeon Graphics; 6 núcleos físicos, 12 procesadores lógicos; GCC/G++ 13.3.0.

Medición: N=5000, 10 pasos, semilla=42, tres repeticiones por configuración.

Objetivo y modelo

Se modelan cuerpos con posición, velocidad y masa. Cada paso suma la aceleración gravitatoria de los demás cuerpos con suavizado $\varepsilon=0.01$ y actualiza primero la velocidad y después la posición (Euler simpléctico, $\Delta t=0.001$). Las tres versiones parten de la misma secuencia SplitMix64 con semilla 42. El tiempo medido por omp_get_wtime() incluye la simulación, y excluye la inicialización y la impresión.

PCAM: Partition

Se eligió partición por datos: el aporte gravitatorio de una pareja de cuerpos es la unidad fina de cálculo potencial durante un paso. La versión directa considera aportes dirigidos (i,j), $j \neq i$; la simétrica considera cada pareja $i<j$ una vez. Las parejas pueden calcularse simultáneamente si sus resultados se acumulan con seguridad. En cambio, el paso t+1 necesita el estado completo de t; los pasos no son independientes. Separar fuerza e integración como fases funcionales no permite ejecutarlas a la vez sobre un mismo paso.

PCAM: Communication

Cada contribución necesita posiciones y masas del estado actual, compartido solo para lectura; los aportes de una misma aceleración deben combinarse. La directa suma esos aportes dentro de la tarea de un cuerpo, con salida exclusiva por hilo. La simétrica aporta a dos cuerpos: guarda sumas en planos privados por hilo y, tras una barrera, reduce por cuerpo antes de integrar. Así evita carreras sin atomics. Al finalizar el paso se sincroniza el estado antes de iniciar el siguiente. Aquí comunicación significa compartir, combinar y sincronizar datos, no enviar mensajes explícitos.

Diseño OpenMP y costos

PCAM: Agglomeration

La unidad fina identificada en P no se lanzó como una tarea OpenMP por pareja. Se agruparon $N-1$ aportes por cuerpo i en la directa y $N-i-1$ parejas por valor de i en la simétrica. Cada iteración exterior es así una tarea práctica con suficiente trabajo para amortizar la planificación. El chunk de OpenMP puede agrupar además varias iteraciones exteriores en una asignación: se midieron 8, 32, 64 y default. Cambia la granularidad de asignación, no la partición matemática inicial. Con static, el chunk explícito también cambia qué bloques recibe cada hilo; el experimento no separa A de M.

PCAM: Mapping

Las tareas exteriores agrupadas se asignan a 2, 4 u 8 hilos. Static predetermina el reparto; dynamic entrega el siguiente chunk disponible; guided reduce progresivamente el tamaño de los bloques. La directa tiene casi $N-1$ interacciones por iteración, mientras que en la simétrica el trabajo cae de $N-1$ a cero, lo que puede causar espera con static. La elección equilibra trabajo ocioso y overhead de scheduling. Se midieron los tres esquemas en la versión directa con 8 hilos; en la simétrica solo se midió static para la comparación principal. Los tiempos no aíslan por sí solos la causa de sus diferencias.

Concurrencia y memoria

Si dos hilos actualizasen directamente la aceleración de i y j, habría escrituras concurrentes. Los planos privados eliminan esa carrera y evitan atomics por par. Su costo es O(hilos × N) memoria por paso, más una reducción O(hilos × N). Las tres versiones conservan complejidad asintótica O(pasos × N²); la versión por pares evalúa N(N−1)/2 parejas en vez de N(N−1) interacciones dirigidas.

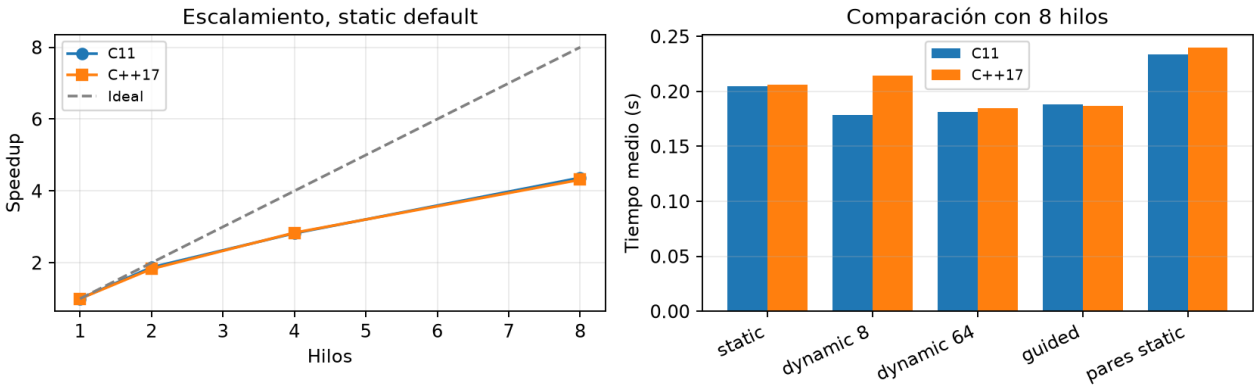
Método de medición

Cada configuración se ejecutó tres veces en el mismo equipo. Se guardó cada tiempo en results_raw.csv o results_cpp_raw.csv y se calculó media y desviación muestral en los CSV resumidos. Speedup S(p)=T□/Tp; efficiency E(p)=S(p)/p, usando hilos reales observados. Las mediciones C y C++17 son corridas separadas; no deben compararse como un experimento apareado.

Resultados medidos: C11

N=5000 · 10 pasos · semilla 42 · tres repeticiones por configuración. Valores redondeados para lectura; los CSV conservan más cifras.

Configuración	Media (s)	Desv. (s)	Speedup	Efficiency
Secuencial (1)	0.8946	0.0070	1.00	1.000
Directa 2 / static	0.4775	0.0223	1.87	0.937
Directa 4 / static	0.3171	0.0182	2.82	0.705
Directa 8 / static	0.2048	0.0005	4.37	0.546
Directa 8 / static 8	0.2082	0.0026	4.30	0.537
Directa 8 / static 32	0.2099	0.0036	4.26	0.533
Directa 8 / static 64	0.2086	0.0012	4.29	0.536
Directa 8 / dynamic 8	0.1789	0.0025	5.00	0.625
Directa 8 / dynamic 64	0.1811	0.0028	4.94	0.618
Directa 8 / guided	0.1880	0.0050	4.76	0.595
Pares 8 / static	0.2340	0.0064	3.82	0.478



La directa con 8 hilos y static default alcanzó 4.37× (eficiencia 0.546). Directa 8 / dynamic 8 obtuvo el menor tiempo medio de esta corrida: 0.1789 s. Los tres chunks static medidos permiten evaluar la granularidad, pero tres muestras no demuestran un orden universal. La versión por pares con static default tardó 0.2340 s, 1.14 veces el tiempo de la directa equivalente.

Resultados medidos: C++17 y corrección

Configuración	Media (s)	Desv. (s)	Speedup	Efficiency
Secuencial (1)	0.8892	0.0045	1.00	1.000
Directa 2 / static	0.4862	0.0242	1.83	0.914
Directa 4 / static	0.3135	0.0194	2.84	0.709
Directa 8 / static	0.2062	0.0029	4.31	0.539
Directa 8 / static 8	0.2094	0.0015	4.25	0.531
Directa 8 / static 32	0.2166	0.0065	4.11	0.513
Directa 8 / static 64	0.2346	0.0176	3.79	0.474
Directa 8 / dynamic 8	0.2141	0.0167	4.15	0.519
Directa 8 / dynamic 64	0.1850	0.0045	4.81	0.601
Directa 8 / guided	0.1869	0.0029	4.76	0.595
Pares 8 / static	0.2396	0.0045	3.71	0.464

Con C++17, static default obtuvo 4.31× con 8 hilos; Directa 8 / dynamic 64 obtuvo el menor tiempo medio (0.1850 s). La versión por pares con static default tardó 0.2396 s frente a 0.2062 s de la directa equivalente. Menos evaluaciones de pares no implicó menos tiempo total: los acumuladores, su inicialización y reducción, y el desequilibrio potencial agregan costo.

Granularidad y asignación

En C11, los chunks static 8, 32 y 64 dieron 0.2082, 0.2099 y 0.2086 s: no se aprecia una ventaja clara entre esos tamaños. En C++17 dieron 0.2094, 0.2166 y 0.2346 s; el chunk 64 fue el más lento de los tres en esta corrida. El chunk agrupa tareas de cuerpos (A), pero con static también cambia su reparto entre hilos (M). Con 8 hilos, la política más rápida medida fue dynamic 8 en C11 (0.1789 s) y dynamic 64 en C++17 (0.1850 s); guided dio 0.1880 y 0.1869 s, y static default 0.2048 y 0.2062 s, respectivamente. Con tres repeticiones no se puede atribuir toda diferencia a la política ni afirmar que el mismo ajuste ganará en otros equipos.

Corrección numérica

Para N=5000 y 10 pasos, se compararon todas las coordenadas, velocidades y masas contra la versión secuencial. Diferencia absoluta máxima: directa=0 en C y 0 en C++17; por pares=1.066e-13 en C y 1.066e-13 en C++17. El checksum impreso fue 4416.553277470315 en las 66 mediciones, pero la comparación de estado completo es la evidencia más fuerte.

Conclusiones

La paralelización directa sigue mejorando hasta 8 hilos, pero ya es sublineal con 2: en C11 los speedups static son 1.87×, 2.82× y 4.37× para 2, 4 y 8 hilos (eficiencias 0.937, 0.705 y 0.546); en C++17 son 1.83×, 2.84× y 4.31× (0.914, 0.709 y 0.539). Al pasar de 4 a 8 hilos el rendimiento mejora solo 1.55× en C11 y 1.52× en C++17, frente al 2× ideal. No se esperan ocho veces más rendimiento al pasar de 8 a 64 núcleos: la eficiencia ya cae con más hilos y persisten sincronización, tráfico de memoria y costos de planificación. Esta extrapolación no se midió; el equipo utilizado tiene 6 núcleos físicos.

Evidencia de compilación y ejecución

Las capturas muestran ejecuciones adicionales con la misma carga y semilla. Sus tiempos individuales pueden diferir de las medias de las tres repeticiones registradas en los CSV.

Compilación C/C++17

```
$ wsl.exe -d Ubuntu-24.04 -- make -s -B all
Compilación completada. Ejecutables generados:
-rwxrwxrwx 1 dom dom 23K Oct 10  2026 build/cpp_nbody_optimized
-rwxrwxrwx 1 dom dom 22K Oct 10 22:03 build/cpp_nbody_parallel
-rwxrwxrwx 1 dom dom 18K Oct 10 22:03 build/cpp_nbody_seq
-rwxrwxrwx 1 dom dom 22K Oct 10 22:03 build/nbody_optimized
-rwxrwxrwx 1 dom dom 22K Oct 10 22:03 build/nbody_parallel
-rwxrwxrwx 1 dom dom 18K Oct 10 22:03 build/nbody_seq
Exit code: 0
PS C:\Users\David\Documents\2026\S2\PARALELA\labPCAM>
```

Ejecución secuencial C++17

```
$ wsl.exe -d Ubuntu-24.04 -- build/cpp_nbody_seq 5000 10
Students: David Dominguez (23712), Jonathan Diaz (23873), Luis Padilla (23663)
Mode: sequential
N: 5000 | Steps: 10 | Seed: 42
Elapsed: 0.933474757 s
Checksum: 4416.553277470315
Processors reported by OpenMP: 12
Maximum threads reported by OpenMP: 12
Exit code: 0
PS C:\Users\David\Documents\2026\S2\PARALELA\labPCAM>
```

Ejecución paralela directa C++17

```
$ wsl.exe -d Ubuntu-24.04 -- build/cpp_nbody_parallel 5000 10 8 static
Students: David Dominguez (23712), Jonathan Diaz (23873), Luis Padilla (23663)
Mode: parallel-direct
N: 5000 | Steps: 10 | Seed: 42
Threads requested: 8 | Threads used: 8
Schedule: static | Chunk: default
Elapsed: 0.222941989 s
Checksum: 4416.553277470315
Processors reported by OpenMP: 12
Maximum threads reported by OpenMP: 12
Exit code: 0
PS C:\Users\David\Documents\2026\S2\PARALELA\labPCAM>
```

Ejecución por pares C++17

```
$ wsl.exe -d Ubuntu-24.04 -- build/cpp_nbody_optimized 5000 10 8 static
Students: David Dominguez (23712), Jonathan Diaz (23873), Luis Padilla (23663)
Mode: parallel-optimized
N: 5000 | Steps: 10 | Seed: 42
Threads requested: 8 | Threads used: 8
Schedule: static | Chunk: default
Elapsed: 0.255957696 s
Checksum: 4416.553277470315
Processors reported by OpenMP: 12
Maximum threads reported by OpenMP: 12
Exit code: 0
PS C:\Users\David\Documents\2026\S2\PARALELA\labPCAM>
```

Hardware

```
$ wsl.exe -d Ubuntu-24.04 -- lscpu

Architecture:                         x86_64
CPU(s):                               12
Model name:                           AMD Ryzen 5 4600H with Radeon Graphics
Thread(s) per core:                   2
Core(s) per socket:                   6
Socket(s):                            1
$ wsl.exe -d Ubuntu-24.04 -- gcc -dumpfullversion
13.3.0
$ wsl.exe -d Ubuntu-24.04 -- g++ -dumpfullversion
13.3.0
Exit code: 0


PS C:\Users\David\Documents\2026\S2\PARALELA\labPCAM>
```